



Python: fonctions

Juan-Manuel Torres

Juan-manuel.torres@univ-avignon.fr

LIA Avignon

Abstraction

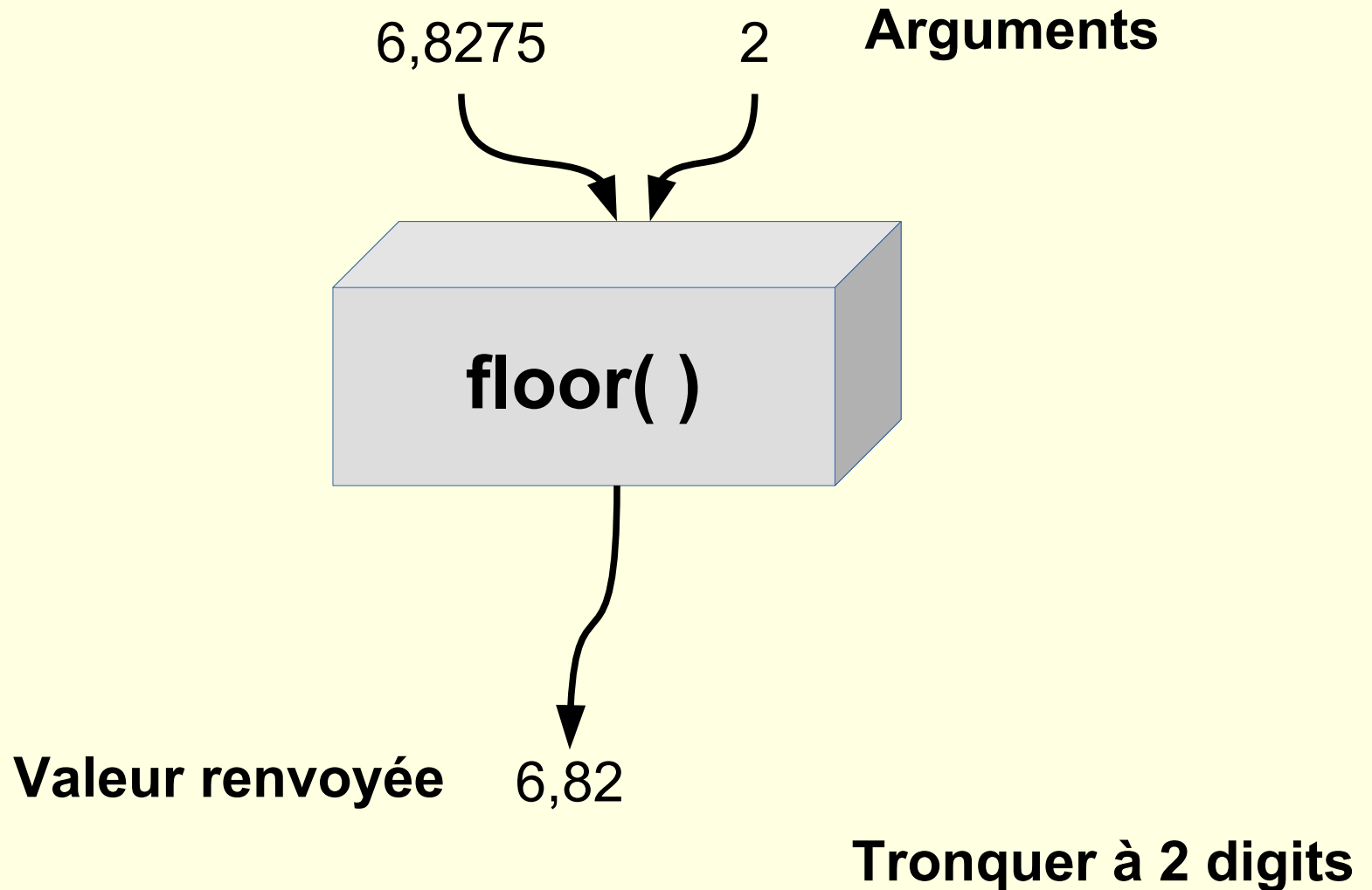
- Technique qui permet de réfléchir à un problème en plusieurs niveaux
- Diviser et conquérir:
 - Un problème est divisé en plusieurs sous-problèmes
 - Les solutions des sous-problèmes combinées peuvent résoudre un grand problème

Programmation structurée

Discipline de programmation qu'incorpore le principe de “Diviser et Conquerir”

- Programmes divisés en sous-programmes
 - Les sous-programmes sont invoqués par leur **nom** et une **liste de variables**
 - Le même sous-programme peut être utilisé pour résoudre des problèmes de la même nature mais avec des valeurs spécifiques différentes
 - Les résultats d'un sous-programme peuvent être combinés avec ceux d'autres sous-programmes

Fonctions en python



Sous-programmes python: fonctions

- En Python, les sous-programmes s'appellent **fonctions**
`def nom (arg, arg, ... arg):`

`instruction`

`. . .`

`instruction`

- où:

- *nom* de la fonction
- *args* sont les arguments de la fonction
 - Une fonction peut avoir 0, 1 ou plus arguments
- *instructions* sont exécutées si la fonction est appelée

Fonctions en python

Les fonctions servent à encapsuler le code

Elles peuvent ou pas avoir des paramètres d'entrée

Définition : les fonctions doivent être définies
préalablement

Appel : Une fonction, est exécutée uniquement au moment
de son appel. Pour appeler une fonction, il suffit d'indiquer
son nom

Fonctions en python

Définition

```
def nom_fonction():  
    print "Hola"
```

Appel

```
nom_fonction()
```

Exécution

```
Hola
```

Résultat d'une fonction

- Une fonction calcule 1 ou plusieurs valeurs
- Renvoyer le resultat d'une fonction : `return`
- Dès que la commande `return`, est trouvée, la fonction se termine et renvoie le contrôle au programme qui l'a appelée
- Si une fonction se termine et aucune valeur a été définie a être renvoyée, la valeur renvoyée sera `None`

Example

```
def f():  
    return
```

```
f()  
None
```

```
def f():  
    return "Salut"
```

```
f()  
Salut
```

```
def f(nom):  
    return "Salut, ", nom, "!"
```

```
f("Juan-Manuel")  
Salut, Juan-Manuel!
```

Valeur renvoyée

Si on veut qu'une fonction retourne une donnée, on utilise **return**

La valeur peut être utilisée pour l'assigner à une variable :

```
def somme(a,b):    # Définition de la somme
    return a+b
```

```
x,y=3,4
s = somme(x,y)    # Appel et assignation
print s
```

Variables locales et globales

- Les variables définies dans une fonction sont **locales**:
seulement peuvent être utilisées DANS la fonction
- Les variables définies en dehors des fonctions sont **globales**:
 - Une variable globale peut être utilisée dans une fonction

Arguments de fonctions

- Les arguments (ou paramètres) sont variables qui reçoivent leurs valeurs initiales à partir du programme appelant
- Les arguments sont des ***variables locales***
 - Les variables locales sont éliminées quand la fonction se termine
 - Si une fonction définit n arguments, l'appel doit inclure n valeurs

Exemple

```
def f(x):                                # x est une variable locale
    return x*x
```

```
print f(10)
100
```

```
print x
```

```
.....
NameError: name 'x' is not defined
```

```
print f()                                # f a besoin d 1 argument
```

```
.....
TypeError: f() takes exactly 1 argument (0 given)
```

Somme d'entiers (while)

$$\text{Somme}(n) = 1 + 2 + 3 + \dots + n$$

Par exemple si $n=5$:

$$\text{Somme}(5) = 1 + 2 + 3 + 4 + 5 = 15$$

- Écrire un programme qui utilise une fonction **itérative** **non déterministe** pour calculer la somme de n entiers

Somme d'entiers

Version non déterministe

def somme(n):

sum = 0 # initialise sum et compteur

i = 1 # sum,i = 0,1

while i <= n:

sum = sum + i # sum += i

i = i+1 # i += 1 mise a jour compteur

return sum

#----- Programme Principale

n = input("Entrez n? ")

print "Somme(",n, ")=", somme(n)

Somme d'entiers (for)

$$\text{Somme}(n) = 1 + 2 + 3 + \dots + n$$

Écrire un programme pour cette somme, qui utilise une fonction **itérative déterministe** pour la calculer

Somme d'entiers

Version deterministe

def somme(n):

sum = 0 # initialise sum

for i in range(1,n+1):

sum = sum + i #sum+=i

return sum

#----- Programme Principale

n = input("Entrez n? ")

print "Somme(",n, ")=", somme(n)

Factorielle

$$\text{Factorielle}(n) = 1 * 2 * 3 * \dots * n = n !$$

Par exemple :

- $\text{Factorielle}(5) = 1 * 2 * 3 * 4 * 5 = 120$
- Écrire un programme qui utilise une fonction **itérative** **déterministe** pour calculer $n !$

Factorielle itérative

```
def fact(x):  
    f = 1  
    for i in range(1,x+1):  
        f = f * i           # 1*2*3* ... *x  
    return f
```

```
#----- Programme principale  
num = input("Donnez n? ")  
if num >= 1:  
    print num, "! = ", fact(num)
```

juanmanuel@karellen-xps: ~/Documents/ENSEIGNEMENT/AVIGNON/BASES_PROGRAMMA

```
juanmanuel@karellen-xps:~/Documents/ENSEIGNEMENT/AVIGNON/BASES_PROGRAMMATION_CMI_Geographie/2016/TD$ python factorielle_for.py
```

Donnez n? 5

5 ! = 120

```
juanmanuel@karellen-xps:~/Documents/ENSEIGNEMENT/AVIGNON/BASES_PROGRAMMATION_CMI_Geographie/2016/TD$ python factorielle_for.py
```

Donnez n? 10

10 ! = 3628800

```
juanmanuel@karellen-xps:~/Documents/ENSEIGNEMENT/AVIGNON/BASES_PROGRAMMATION_CMI_Geographie/2016/TD$ python factorielle_for.py
```

Donnez n? 100

100 ! = 93326215443944152681699238856266700490715968264381621468592963895217599993229915608941463976156518286253697920827223758251185210916864000000000000000000000000

```
juanmanuel@karellen-xps:~/Documents/ENSEIGNEMENT/AVIGNON/BASES_PROGRAMMATION_CMI_Geographie/2016/TD$
```

Factorielle recursive

On sait que : $N! = 1 \times 2 \times 3 \times \dots \times N$

Mais : $N! = N \times (N-1)! ; 1! = 1$

$5! = 5 \times 4! ; 4! = 4 \times 3! ; 3! = 3 \times 2! ; 2! = 2 \times 1! ; 1! = 1$

Ainsi $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Modifier le programme pour utiliser une fonction **récursive** pour calculer $n!$



Factorielle récursive

```
def fact(x):  
    if x == 1:  
        return 1  
    else:  
        return x * fact(x-1)
```

```
#----- Programme principale  
num = input("Donnez n? ")  
if num >= 1:  
    print num, "! = ", fact(num)
```



Puissance x^y

Function puissance

```
def puissance(x,y):  
    return x**y
```

#----- Programme principale

```
base = input("Base? ")  
terms = input("Combien de termes? ")
```

Resultat

```
for i in range(terms+1):  
    print base,"^",i," = ",puissance(base,i)
```

juanmanuel@karellen-xps: ~/Documents/ENSEIGNEMENT/AVIGNON/BASES_PROGRAMMA

juanmanuel@karellen-xps:~/Documents/ENSEIGNEMENT/AVIGNON/BASES_PROGRAMMATION_CMI_Geographie/2016/TD\$ python puissance.py

Base? 2

Combien de termes? 10

$2^0 = 1$

$2^1 = 2$

$2^2 = 4$

$2^3 = 8$

$2^4 = 16$

$2^5 = 32$

$2^6 = 64$

$2^7 = 128$

$2^8 = 256$

$2^9 = 512$

$2^{10} = 1024$

juanmanuel@karellen-xps:~/Documents/ENSEIGNEMENT/AVIGNON/BASES_PROGRAMMATION_CMI_Geographie/2016/TD\$

Fibonacci sans fonction...

Suite de n premier termes de Fibonacci

n = input("Limite? ")

a,b=0,1

for i in range(1,n+1):

print b

a,b=b,a+b

Fibonacci itératif avec fonction

Suite de Fibonacci (fonction)

#----- Fonction Fibonacci

def fibo(n):

a,b=0,1

for i in range(1,n+1):

print i,"": ",b

a,b=b,a+b

#----- Programme principal

lim = input("Limite? ")

fibo(lim)

Fibonacci réursive

#----- Fonction Fibonacci réursive

def fibo(n):

if n == 0:

return 0

elif n == 1:

return 1

else:

return fibo(n-1)+fibo(n-2)

#----- Programme principal

lim = input("Limite? ")

for i in xrange(1,lim+1):

print i,":",fibo(i)



Liste variable de paramètres

- Si le dernier argument d'une definition de fonction commence par *, les valeurs passées sont plusieurs
- **len(liste)** donne la TAILLE de la liste
- Ex.:

```
>>> def moyenne(*valeurs):  
    total=0.0  
    for x in valeurs:  
        total += x  
    return total/len(valeurs)
```

```
...  
>>> moyenne (1,2,3,4)  
2.5
```



Maison

- 1- Définir une fonction **max()** qui prendra comme argument 2 numéros et retourne le plus grand d'eux (sans utiliser la fonction max prédéfinie de python)
- 2- Définir une fonction **max_de_3()** , qui reçoit trois valeurs comme arguments et renvoie le plus grand d'entre eux



Puissance récursive

La fonction puissance(b,n), peut être calculé avec n itérations pour obtenir la valeur b^n

On peut, faire différemment en optimisant le calcul si l'on considère que:

$$b^n = b^{(n/2)} \times b^{(n/2)}$$

si n est pair

$$b^n = b^{(n-1)/2} \times b^{(n-1)/2} \times b$$

si n est impair



Puissance récursive...

Dans une fonction récursive, on doit décider quel est la cas base et quel est le cas récursif.

Dans le cas de la puissance, nous prendrons $n = 0$ comme le cas base, et la fonction doit renvoyer 1, car :

$x^0 = 1$ pour tout x (différent de 0)

Le cas récursif aura 2 parties, correspondantes aux deux possibles groupes de valeurs de n : **pair** ou **impair**



Fonction python puissance récursive

```
def puissance(b,n):  
    if n <= 0:  
        return 1                # Cas base  
    if n % 2 == 0:                # n pair  
        p = puissance(b, n/2)  
        return p * p  
    else:                        # n impair  
        p = puissance(b, (n-1)/2)  
        return p * p * b
```



Fonction python puissance récursive

Exécution :

```
>>> puissance(2, 10)
```

```
1024
```

```
>>> puissance(3, 3)
```

```
27
```

```
>>> puissance(5, 0)
```

```
1
```



Fonction python puissance récursive : trace

```
puissance(2,10)
```

```
    p = puissance(2,5)
```

```
        p = puissance(2,2)
```

```
            p = puissance(2,1)
```

```
                p = puissance(2,0)
```

```
                    return 1
```

```
                return 1 * 1 * 2
```

```
            return 2 * 2
```

```
        return 4 * 4 * 2
```

```
    return 32 * 32
```

```
# b → 2 n → 10
```

```
# b → 2 n → 5
```

```
# b → 2 n → 2
```

```
# b → 2 n → 1
```

```
# b → 2 n → 0
```

```
# b → 2 n → 1 p → 1
```

```
# b → 2 n → 2 p → 2
```

```
# b → 2 n → 5 p → 4
```

```
# b → 2 n → 10 p → 32
```



Attention !
Contrôle écrit

Le vendredi 07 octobre 2016

14h30-16h00 Amphis

Blaise+Ada

Cours théorique + Exercices

